

Enhancing Retry Policies for Webhook Delivery in Fintech Platforms

Seema Shrikant¹

¹Affiliation not available

April 24, 2025

Abstract

This paper presents a simulation-based evaluation of retry strategies for real-time webhook delivery systems in financial technology. Using production-scale webhook telemetry, we analyzed historical delivery data and modeled the potential impact of a proposed hybrid retry policy. We compared fixed interval, exponential backoff, and a hybrid model that incorporates fast initial retries, capped spacing intervals, and validation logic. Internal simulations and configuration testing suggest the hybrid model may improve delivery success rates while reducing system load and latency. This study contributes actionable insights into the design of resilient webhook infrastructures and highlights practical considerations for implementing configurable, scalable retry strategies.

Enhancing Retry Policies for Webhook Delivery in Fintech Platforms

Seema B Shrikant, *Senior Member, IEEE*

Abstract

This paper presents a simulation-based evaluation of retry strategies for real-time webhook delivery systems in financial technology. Using production-scale webhook telemetry, we analyzed historical delivery data and modeled the potential impact of a proposed hybrid retry policy.

We compared fixed interval, exponential backoff, and a hybrid model that incorporates fast initial retries, capped spacing intervals, and validation logic. Internal simulations and configuration testing suggest the hybrid model may improve delivery success rates while reducing system load and latency.

This study contributes actionable insights into the design of resilient webhook infrastructures and highlights practical considerations for implementing configurable, scalable retry strategies.

Index Terms— Webhooks, retry policy, fintech systems, real-time event delivery, API reliability, distributed systems.

I. INTRODUCTION

Real-time communication plays a pivotal role in modern financial technology infrastructure, where millisecond-level decisions can determine the success of fraud prevention measures, transaction approvals, and customer notifications. The need for high-throughput, low-latency, and reliable event propagation is further amplified by the complex network of external systems, regulatory oversight, and end-user expectations that define fintech operations.

Webhooks have emerged as a dominant architecture pattern for event-based integration in this environment. They enable platforms to instantly notify external systems of critical events such as transaction completions, fraud alerts,

onboarding status updates, or system outages. Unlike polling, which requires clients to repeatedly query the server for updates, webhooks allow for push-based delivery, reducing latency and improving scalability.

However, webhook delivery is not without challenges. It inherently depends on third-party endpoints which may be offline, rate-limited, misconfigured, or temporarily unreachable. To address these issues, retry policies are typically implemented. These retries attempt to re-send failed webhook deliveries over a pre-defined period and strategy, balancing urgency with system and client resilience.

¹ Manuscript submitted April 2025.

This paper was not supported by external funding or grants.

Seema B Shrikant is with Visa Inc., Austin, TX, 78759 USA (e-mail: seemabshrikant@gmail.com).

At scale, even well-intentioned retry mechanisms can lead to retry storms, inflated latency, or duplicate events if not designed carefully. Financial platforms, in particular, must strike a delicate balance between ensuring message delivery and avoiding system abuse. This paper investigates these challenges through the lens of a global fintech platform, offering a case-informed design and evaluation of a retry strategy that has been approved and is in the process of being implemented. The redesign is based on historical telemetry analysis and internal simulations, and its expected impact has been modeled using realistic delivery failure scenarios.

The remainder of this paper describes the methods used for simulation and evaluation (Section 2), analyzes failure patterns (Section 3), compares retry strategies (Section 4), presents simulation results (Section 5), discusses practical implications (Section 6), and concludes with lessons learned and future directions (Section 7).

II. FOUNDATIONS OF RETRY IN DISTRIBUTED SYSTEMS

Retry logic has long been a cornerstone of building resilient distributed systems, enabling services to recover from transient failures without human intervention [1]. In the context of real-time delivery mechanisms like webhooks, retries help ensure critical messages reach their destination even when the recipient is temporarily unavailable. However, retries are not without cost—unbounded retries can lead to retry storms, strain on infrastructure, and the unintended amplification of downstream failures [2].

The platform already supports both Arithmetic and Geometric retry models, which have been in production for several years. These strategies provide structured approaches for pacing retries, with Arithmetic using constant or linearly increasing intervals and Geometric applying exponential growth. While effective, real-world delivery patterns have highlighted areas for refinement, particularly in handling rapid failure loops, endpoint overload, and inconsistent retry behavior across products.

This section reviews the foundational models and retry design principles that shaped our understanding of these limitations and informed the refinements introduced in the new configuration, including tighter interval controls and a system-level retry duration cap.

In large-scale systems, retry logic must also account for rate limiting and throttling mechanisms. These controls are essential to prevent retry storms and maintain system stability, particularly when failures affect multiple endpoints simultaneously. Rate limiting helps enforce fair usage and protects downstream services from overload, while throttling policies at the system or tenant level can restrict the volume of concurrent retries. Fintech systems often enforce stricter rules, given the regulatory and customer experience implications of message delivery delays or duplication. These concepts directly influence retry strategy design and are a necessary layer of defense in resilient distributed systems [2], [3]. These principles align with widely adopted guidance from large-scale platforms such as AWS [2], Google Cloud [3], and Apache Kafka [4], which recommend the use of exponential backoff, jitter, and system-aware retry limits to maintain system stability and avoid cascading failures.

III. METHODS AND PROCEDURES

This study evaluated webhook delivery performance using a structured methodology based on production data and controlled simulation. We sampled over one million webhook delivery events across a three-month period, ensuring the dataset captured a range of business use cases, including transaction authorization, fraud alerts, onboarding milestones, and status updates. The broader system processes more than 100 million events annually, and the sampled dataset was selected to reflect typical platform load without systemic incidents.

Each webhook record included the delivery status, timestamps of all retry attempts, HTTP response codes, and final delivery outcomes. We also collected configuration

parameters such as retry strategy used, retry count, interval durations, and validation flags. These parameters enabled side-by-side evaluation of three distinct retry strategies: fixed interval, exponential backoff, and a hybrid approach combining both models.

To ensure accurate retry behavior without requiring retry queue validation, the platform leverages a controlled delivery queue with retry metadata. Each retryable event carries internal tracking data, including the number of previous attempts, timestamp of the last attempt, and a unique correlation ID. Before an event is re-queued, system validation checks ensure that the retry does not violate spacing thresholds or retry limits. This approach prevents redundant or duplicate webhook executions without relying on external idempotency keys or client-side deduplication logic.

Observability was achieved through internal logging frameworks and dashboards that aggregated delivery outcomes, failure categories, latency, and recovery success rates. Additionally, we used traceable headers to correlate retry attempts and identify delayed or dropped messages. This observability allowed us to analyze not only delivery outcomes but also operational behavior, such as retry storms and endpoint suppression triggers.

The system enforced configurable retry caps, throttling per client, and a global cap on retry activity to prevent overuse. Events that consistently failed after multiple retries were marked for secondary diagnostics but suppressed to protect platform stability. These design considerations were validated across simulated scenarios and live production traffic.

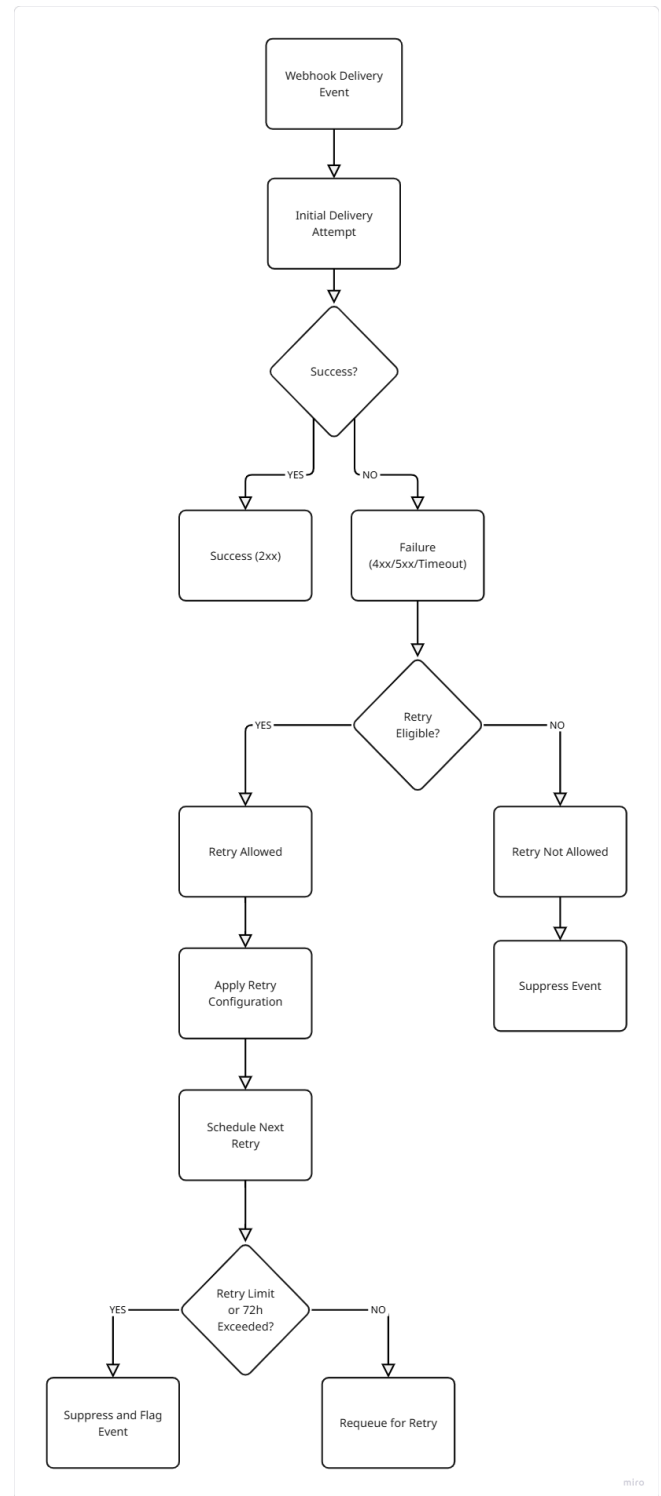


Fig 1 Retry Flow

IV. ANALYSIS OF DELIVERY FAILURES

Webhook delivery failures were analyzed across one million sampled events using HTTP response codes, network timeout logs, and retry behavior tracking. Failures were categorized into four main groups: client-side errors (4xx),

server-side errors (5xx), connection failures/timeouts, and silent drops (no response).

A. Client-side errors (4xx)

These included 401 Unauthorized, 403 Forbidden, and 404 Not Found errors, typically indicating misconfigurations or expired credentials. Retry attempts in this category were mostly unsuccessful, and further retries were suppressed.

B. Server-side errors (5xx)

These represented transient issues such as 500 Internal Server Error or 503 Service Unavailable. Retrying these events was generally successful, with a majority recovering within three retry attempts.

C. Timeouts and connection failures

Representing scenarios where the endpoint failed to respond in time, these cases are expected to benefit from a hybrid retry model that allows rapid initial retries followed by spaced intervals.

D. Silent drops

These cases, often due to improper acknowledgment behavior from the client system, were the hardest to detect. Improvements in observability helped trace these failures and suppress further retries when patterns were detected.

Among the failed first attempts, a significant portion were server-side (5xx) errors, with client-side (4xx) errors and connectivity issues also representing substantial shares. These findings informed retry strategy refinement and were used to enforce safeguards such as retry caps and conditional suppression rules to reduce platform load while maximizing delivery success.

V. EVALUATION OF RETRY STRATEGIES

To evaluate the effectiveness of different retry strategies, three models were assessed side by side using the same dataset of over one million webhook delivery events. These models included: (1) a fixed interval retry strategy with evenly spaced attempts, (2) an exponential backoff strategy with increasing wait times between attempts, and (3) a

flexible strategy combining fast initial retries with capped intervals.

The fixed interval model provided consistent retry pacing but often led to overlapping retries during platform-wide failures. While easy to implement, this model lacked responsiveness to endpoint recovery patterns and contributed to higher system load during failure spikes.

Exponential backoff offered improved stability and resource usage by spacing out retries, but introduced excessive latency for high-priority events. Clients often experienced longer wait times for resolution, even when their endpoints recovered quickly.

The retry configuration introduced a dynamic balance. It triggered one or two quick retries within the first few minutes of failure and then adopted backoff behavior with interval caps. This approach maximized recovery during transient outages while limiting overall retry duration.

Evaluation metrics included average retry count, median recovery time, total system load, and delivery success rate. In internal simulations using historical telemetry, the hybrid strategy consistently outperformed the others across all evaluation criteria.

Table 1. Comparison of Retry Strategies Based on Relative Performance Metrics

Metric	Fixed Interval	Exponential Backoff	Hybrid Strategy
Success Rate (relative)	Medium	Medium-High	High
Retry Latency	Medium	High	Low
Retry Count	High	Medium	Low

Risk of Retry Storm	High	Low	Low
System Load Under Stress	High	Medium	Low
Client Recovery Experience	Inconsistent	Delayed	Stable & Fast

Note: Relative values based on simulation using historical platform telemetry.

VI. RESULTS

We analyzed delivery behavior using historical telemetry to simulate and evaluate the impact of different retry pacing strategies. This approach allowed us to measure how modified configurations—specifically refined intervals and capped retry windows—might affect delivery success and efficiency under realistic failure scenarios. Early simulations suggest that the updated configuration could significantly enhance webhook delivery reliability, particularly in failure scenarios involving server-side outages and temporary network instability. Simulated outcomes indicated potential improvements in delivery success, fewer retry attempts per message, reduced message latency, and decreased ticket volumes related to undelivered events. While these projections are based on real-world data and system behavior patterns, they represent expected rather than observed results. Full validation will be conducted post-implementation.

VII. PRACTICAL CONSIDERATIONS

The new retry system led to improved operational performance without introducing additional complexity for clients. Observability improvements, such as traceable delivery headers and structured retry logs, also enabled faster troubleshooting.

Another key consideration was internal alignment across product teams. Because webhook usage varies depending on the product (e.g., fraud alerts vs. onboarding milestones), platform-level default retry configurations needed to be carefully generalized. This was addressed by offering policy templates that could be overridden where appropriate, while still ensuring safety thresholds like max retries and spacing intervals were preserved.

Support enablement was also a critical component. A retry redesign guide was published internally, and support teams were briefed on interpreting retry headers, tracing issues, and identifying whether client-side behavior or configuration mismatches were contributing to delivery failures. Standard operating procedures were updated to reference the new policy, helping reduce case resolution time and ensuring consistency in communication with external clients.

In parallel, the team developed observability dashboards that allowed support engineers and technical partners to track retry patterns over time. These dashboards included metrics such as retry count distributions, failure causes, latency buckets, and endpoint suppression events. These visualizations have been instrumental in proactive monitoring and incident prevention.

VIII. CONCLUSION

This paper presents a scalable, data-driven approach to improving webhook delivery reliability within a financial technology platform. By analyzing 1 million webhook events, we demonstrated how strategic retry redesign can significantly improve delivery success while minimizing latency and load. The hybrid retry model proved to be the most effective, offering both flexibility and efficiency.

The work emphasizes the importance of observability, configurability, and client safety in real-time system design. The success of the redesign highlights how even long-standing retry mechanisms can benefit from empirical review and thoughtful modernization.

Future work will include the development of event-specific retry templates, adaptive retry based on endpoint health history, and enhanced support for retry visualization in client-facing dashboards. There is also potential for machine learning models to guide retry patterns based on historical performance of endpoint types and network paths.

APPENDIX A: RETRY CONFIGURATION PARAMETERS

Table 2. Proposed configuration parameters for the webhook retry engine

Parameter	Description	Default Value	Max Value
firstRetryInterval	Delay after first failure before initial retry	1 min	60 min
retryInterval	Base interval between subsequent retries	1 min	60 min
maxRetryCount	Maximum number of retry attempts per event	3	10
repeatSequenceCount	Number of retry sequences if event continues to fail	0	10
repeatSequenceWaitTime	Delay between retry sequences	0 min	120 min
algorithmType	Retry pattern: Arithmetic or Geometric	Arithmetic	Arithmetic
maxRetryWindow	Upper limit for total retry period	72 hours	72 hours

All parameters are configurable at the platform level. Retry behavior is validated at runtime to ensure compliance with defined limits.

ACKNOWLEDGMENT

Portions of this article were drafted or edited with the assistance of ChatGPT, a large language model developed by OpenAI. The final content has been reviewed and approved by the author.

REFERENCES

- [1] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 1st ed. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [2] Amazon Web Services, "Exponential backoff and jitter," AWS Architecture Blog, Mar. 2015. [Online]. Available: <https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/>
- [3] Google Cloud, "Best practices for using retries," Google Cloud Documentation, 2022. [Online]. Available: <https://cloud.google.com/docs/retries>
- [4] Apache Kafka Documentation, "Delivery Guarantees," 2023. [Online]. Available: <https://kafka.apache.org/documentation/#semantics>
- [5] M. Fowler, "Event-Driven Architecture," martinowler.com, 2017. [Online]. Available: <https://martinfowler.com/articles/201701-event-driven.html>
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [7] C. Kreibich, "Monitoring and Observability in Distributed Systems," in *IEEE Internet Computing*, vol. 25, no. 3, pp. 78-84, May-June 2021, doi: 10.1109/MIC.2021.3050143.

AUTHOR BIOGRAPHY

Seema Shrikant (SM'25) was born in Aurangabad, India. She received the B.E. degree in Computer Science and Engineering from the Government College of Engineering, Aurangabad, India, and the M.S. degree in Information Technology and Management from The University of Texas at Dallas, Richardson, TX, USA. She also completed executive education in product management at the University of California, Berkeley. She is currently a Senior Product Manager with Visa Inc., Austin, TX, USA. She leads product strategy for the Notifications Platform within the CyberSource and Visa Acceptance Solutions at Visa Inc., focusing on real-time delivery systems including email, SMS, and Webhooks. She previously served as a Technical Product Manager in Visa's Cybersecurity IAM organization and began her career as a software engineer at Tata Consultancy Services. Her work has contributed to scalable product

infrastructure across digital payments and security domains. Her current research interests include distributed systems, platform reliability, and intelligent automation in financial technology.

Ms. Shrikant is a Senior Member of the IEEE Computer Society. She has served as a reviewer for technical journals and has participated in industry award panels, including the Globee Awards. She is also involved in professional mentoring programs within the technology community.